Software Architecture Specification (SAS)

Client: University of Pretoria - Biophysics Group

Team Member	Student Number
Kudakwashe Mujuru	u20620111
Carlos Juma	u22602047
Nomzi Phosa	u23577160
Mbali Thobejane	u23543672

Team: CoreTech

Contact: coretech.capstone@gmail.com

Date of submission: 04/09/2026

Contents Page

Contents

1. Introduction

1.1 Purpose

This Software Architecture Specification (SAS) document defines the architectural structure, design decisions, and deployment strategy for the Full SMS system. It describes how the system is organized to satisfy the functional and non-functional requirements outlined in the Software Requirements Specification (SRS).

1.2 Scope

Full SMS Web is a browser-based scientific data analysis platform designed to improve accessibility to single-molecule spectroscopy (SMS) analysis tools. The system enables biophysics researchers to upload, analyze, visualize, and share fluorescence measurement data from any location without requiring local software installation or high-performance workstations. Additionally, the system allows researchers to extend functionality through a plugin system.

1.3 Architectural Goals

- **Modularity:** Separation of frontend, backend API, and analysis services for independent development and scaling
- **Extensibility:** Plugin architecture allowing researchers to add custom analysis capabilities
- **Performance:** Efficient processing of large scientific datasets with background task execution
- **Security:** Secure authentication, data isolation, and sandboxed plugin execution
- **Reliability:** Consistent data handling with proper error recovery mechanisms

1.4 Architectural Patterns

The Full SMS Web Application architecture is based on several established architectural patterns that work together to provide separation of concerns, scalability, and maintainability. This section describes the high-level patterns that define the system structure.

1.4.1 Layered Architecture

The system follows a strict layered architecture with clear separation between presentation, business logic, and data access layers. Each layer has defined responsibilities and communicates only with adjacent layers.

Layer Structure:

Layer	Technology	Responsibilities
Presentation Layer	Next.js Frontend	User interface, input validation, data visualization, client-side state management
Application Layer	FastAPI Backend	Business logic orchestration, request routing, authentication, task coordination
Service Layer	Analysis Workers	Scientific computation, data processing, plugin execution
Data Access Layer	Supabase Client	Database queries, file storage, caching operations
Data Layer	PostgreSQL + Storage	Persistent data storage, relational data, file objects

The frontend presentation layer interacts exclusively with the backend application layer through REST API calls. It never accesses the database directly. The application layer coordinates between the service layer (for analysis tasks) and the data access layer (for persistence). This separation allows independent development and testing of each layer.

1.4.2 Client-Server Architecture

The system implements a distributed client-server architecture where the client (browser-based frontend) and server (backend API) are physically separated and communicate over HTTP/HTTPS.

Separation Characteristics:

- **Co-located Deployment:** The frontend and backend both run on the same AWS Lightsail instance, with Nginx routing requests based on path (`/` to the frontend, `/api/py/*` to the backend). Each is deployed as an independent process (PM2 for the frontend, Docker for the backend) via the CI/CD pipeline, allowing them to be rebuilt and restarted independently even though they share infrastructure.
- **Technology Independence:** The client uses TypeScript and React while the server uses Python and FastAPI. This separation allows each tier to use the most appropriate technology for its domain.

- **Stateless Communication:** The server maintains no client session state between requests. All necessary context is passed in JWT tokens or request parameters, enabling horizontal scaling of the backend.
- **Multiple Clients:** The architecture supports multiple client types (web browser, desktop application, mobile app) consuming the same backend API without code duplication.

1.4.3 Service-Oriented Architecture

The backend is decomposed into specialized services that communicate through well-defined interfaces. While not a full microservices architecture, the system exhibits service-oriented characteristics.

Service Components:

Service	Deployment	Function
API Service	Docker container (Lightsail)	HTTP request handling, authentication, task dispatch
Worker Service	Docker container (Lightsail)	Analysis task execution, result computation
Message Broker Service	Docker container (Lightsail)	Task queue management, result caching
Plugin Execution Service	AWS Lightsail (internal process, port 8001)	Sandboxed user code execution
Database Service	Supabase managed instance	Data persistence, authentication
Storage Service	Supabase managed storage	File object storage, HDF5 files

Each service can be scaled independently based on load. The API service can run multiple replicas behind a load balancer. Worker services can be added or removed based on queue depth. The plugin execution service runs on separate infrastructure for security isolation.

1.4.4 Event-Driven Architecture

The system uses asynchronous message-driven communication for long-running operations. This prevents blocking and enables responsive user interfaces during compute-intensive analysis tasks.

Event Flow:

1. User initiates analysis operation through frontend UI
2. Frontend sends HTTP request to backend API
3. API validates request and publishes task message to Redis queue
4. API immediately returns task identifier to frontend
5. Celery worker consumes message from queue
6. Worker executes analysis computation

7. Worker publishes result to Redis with task identifier
8. Frontend polls API for task completion status
9. API retrieves result from Redis and returns to frontend
10. Frontend updates UI with analysis results

This pattern decouples the request from the computation. The API service never blocks waiting for analysis to complete. Multiple tasks can execute concurrently across worker processes. Failed tasks can be retried without client involvement.

1.4.5 Repository Pattern

Data access is abstracted through repository-style interfaces that hide the underlying storage implementation from business logic.

Implementation:

The backend defines service modules (workspace_service, session_service, file_service, plugin_service) that encapsulate all data access operations. Controllers never interact with the database directly. Instead, they call service functions that handle the storage details.

For example, saving a session involves:

```
# Controller layer
session_id = create_session_controller(request, user_id)

# Service layer (repository pattern)
def create_session_controller(request, user_id):
    session_data = build_session_state(request)
    return session_service.save_session(session_data, user_id)

# Data access layer
def save_session(session_data, user_id):
    return supabase.table('sessions').insert({
        'user_id': user_id,
        'data': session_data,
        'created_at': datetime.now()
    })
```

This abstraction allows changing the database implementation (PostgreSQL to MongoDB, for example) without modifying controller or business logic code.

1.4.6 Model-View-Controller (MVC) Pattern

The frontend follows a variant of MVC adapted for React's component architecture.

Component Mapping:

- **Model:** React Context providers (AuthContext, Hdf5DataContext, SessionContext) manage application state. This data represents the current state of measurements, analysis results, and user session.
- **View:** React components in the views/ and components/ directories render the UI based on current state. Views are pure functions of props and context, producing

the same output for the same input.

- **Controller:** Event handlers and API client functions (lib/api/) handle user interactions, validate inputs, call backend APIs, and update context state based on responses.

User actions trigger controller functions that call the API and update model state. Context changes trigger view re-renders through React's observer mechanism. This unidirectional data flow prevents inconsistent UI states.

1.4.7 Adapter Pattern (Architectural Level)

The system uses the adapter pattern at the architectural level to integrate the legacy desktop application codebase with the modern web API.

Legacy Integration:

The api/legacy/ directory contains the original scientific analysis code from the desktop application. This code was written for DearPyGui with synchronous file I/O and expects NumPy arrays as input.

The backend API acts as an adapter:

1. Receives HTTP requests with JSON payloads
2. Converts JSON to NumPy arrays and Python objects
3. Calls legacy analysis functions with converted data
4. Converts NumPy results back to JSON-serializable dictionaries
5. Returns HTTP responses to frontend

This allows reuse of validated scientific code without rewriting algorithms in JavaScript or creating language bindings. The research group maintains a single implementation of their analysis methods.

1.4.8 Facade Pattern (Architectural Level)

Complex subsystems are hidden behind simplified interfaces to reduce coupling and improve maintainability.

Example Implementation:

The export service provides a single export_data() function that handles multiple file formats, storage operations, caching, and session retrieval. The controller simply calls this one function and receives a download URL. The complexity of coordinating Redis cache lookups, Supabase storage downloads, format conversion, and multi-file zipping is encapsulated within the service.

Similarly, the frontend API client (lib/api/axiosInstance.ts) provides a facade over the HTTP library. It automatically adds authentication headers, handles token refresh, formats errors, and manages request/response interceptors. Components make simple function calls without dealing with these cross-cutting concerns.

1.4.9 Pattern Summary and Interactions

Pattern	Scope	Primary Benefit
Layered Architecture	System-wide	Separation of concerns, testability
Client-Server	Frontend/Backend	Independent deployment, technology choice
Service-Oriented	Backend components	Independent scaling, fault isolation
Event-Driven	Async operations	Responsive UI, concurrent processing
Repository	Data access	Database abstraction, flexibility
MVC	Frontend components	Predictable state management
Adapter	Legacy integration	Code reuse, single source of truth
Facade	Complex subsystems	Simplified interfaces, reduced coupling

These patterns work together to create a system that is maintainable, scalable, and adaptable to changing requirements. The layered architecture provides structure. Client-server enables distributed deployment. Service-orientation allows independent scaling. Event-driven communication prevents blocking. Repository pattern abstracts data access. MVC organizes frontend code. Adapter integrates legacy code. Facade simplifies complex operations.

1.5 Design Patterns

The Full SMS Web Application employs several well-established design patterns to ensure maintainability, extensibility, and separation of concerns. These patterns address recurring architectural challenges in session management, data export, service initialization, legacy code integration, and dynamic UI rendering.

1.5.1 Memento Design Pattern

The Memento Design Pattern is used to implement the Save and Load Session functionality, enabling users to capture and restore the complete state of their analysis work.

Pattern Components:

Pattern Role	Implementation	Responsibility
Memento	Analysis Session	Contains a state attribute with raw data captured when user saves a session
CareTaker	Recent Sessions List	Stores collection of all saved sessions for retrieval
Originator	Analysis Hub	Creates and sets sessions by updating parameters

Key Methods:

- **saveSession():** Captures the current state of the analysis session including all loaded measurements, computed levels, clustering results, and fit parameters. Stores this state in the memento object.
- **loadSession():** Retrieves the state of a specific saved session from the CareTaker and restores it to the Analysis Hub, reconstructing the user's previous work environment.
- **createSession():** Originator method that initializes a new analysis session and returns a memento representing its initial state.
- **setSession():** Originator method that accepts a memento and restores the Analysis Hub to the captured state by updating all relevant parameters.

Rationale: This pattern allows users to save intermediate analysis states without exposing the internal structure of the Analysis Hub. Sessions can be shared, versioned, and restored without violating encapsulation, and the Analysis Hub remains independent of storage mechanisms.

1.5.2 Export Tab Design Patterns

The export functionality implements three complementary design patterns to manage the complexity of generating multiple data formats from various analysis results.

Facade Pattern

Description: Provides a single, simplified interface to a set of complex subsystems, hiding their internal details from the client.

Implementation: The `export_service.export_data()` function sits in front of `storage_service`, `session_service`, `redisClient`, and the legacy `io.exporters` / `plot_exporters` modules. The controller calls one function, `export_data(request, user_id)`, and never has to know how caching, storage downloads, session lookups, or file formatting actually happen underneath.

Rationale: The export feature touches multiple moving parts (Redis cache, Supabase storage, saved analysis sessions, multiple export formats). Wrapping all of that behind one service call keeps `export_controller.py` thin and means the controller only has to handle HTTP concerns (status codes, exceptions), not orchestration logic.

Template Method Pattern

Description: Defines the skeleton of an algorithm in one place, with interchangeable steps for the varying parts.

Implementation: The functions `export_intensity_trace()`, `export_levels()`, `export_groups()`, and `export_fit_results()` all follow the exact same structure:

1. Build the output path
2. Ensure the directory exists
3. Branch on format to call one of four private per-format functions
4. Log the operation
5. Return the path

The skeleton is identical across all four public functions, only the per-format helper functions and their inner logic differ.

Rationale: Four different data types (intensity traces, levels, groups, fit results) need export support across four formats each (CSV, Parquet, Excel, JSON). Repeating the standard workflow in one place per data type avoids rewriting that scaffolding sixteen times and ensures consistent error handling and logging.

Builder Pattern

Description: Constructs a complex object step by step, separating the construction process from the final representation.

Implementation: The `_package_outputs()` function collects a list of `(Path, str)` pairs from each export result and, when there is more than one, zips them together into a single downloadable file with a generated name based on categories and measurement IDs.

Rationale: Users can select multiple export types at once (intensity, levels, groups, plots), so the output needs to be assembled from several pieces into one deliverable. The Builder pattern separates the construction of individual exports from the final packaging step, allowing flexible combinations.

1.5.3 Singleton Pattern

Description: Restricts the instantiation of a class to a single, globally accessible instance.

Implementation: Employed for external service connections, specifically database and cache clients:

- **Backend:** `api/utils/redis_client.py`, `api/utils/supabase_client.py`
- **Frontend:** `frontend/lib/api/axiosInstance.ts`

Each of these modules exports a single instance that is imported throughout the application, ensuring all components share the same connection.

Rationale: Prevents the application from exhausting connection pools by avoiding repeated instantiation of new connections during request lifecycles. Database and HTTP clients maintain internal connection pooling and state that must be shared across the application for optimal performance.

1.5.4 Adapter Pattern

Implementation: The `api/legacy/` directory serves as an adapter layer, including:

- `api/legacy/analysis/` - Scientific analysis algorithms (change point detection, clustering, lifetime fitting, correlation)
- `api/legacy/io/` - HDF5 file reading, session persistence, data exporters

These modules provide FastAPI-compatible interfaces to the original desktop application codebase, applying the same adapter pattern described in Section 1.4.7 at the code level.

1.5.5 Strategy Pattern

Description: Enables the dynamic selection of an algorithm or behavior at runtime based on context.

Implementation: Utilized within the frontend plugin rendering system at `frontend/components/plugins` which includes distinct strategies:

- `HeatmapRenderer.tsx` - Renders 2D heatmap visualizations
- `HistogramRenderer.tsx` - Renders distribution histograms
- `PlotRenderer.tsx` - Renders line and scatter plots
- `TableRenderer.tsx` - Renders tabular data
- `ValueRenderer.tsx` - Renders single scalar values

The parent component selects the appropriate renderer based on the `outputType` field in the plugin execution result.

Rationale: Replaces monolithic conditional logic with interchangeable components, allowing the UI to dynamically render the appropriate visualization based on the specific data output type of a given plugin. This makes it trivial to add new visualization types without

modifying existing renderers.

1.5.6 Provider and Observer Pattern

Description: Establishes a one-to-many dependency where a central subject notifies dependent observers of any state changes.

Implementation: Deeply integrated into the React frontend via the Context API at `frontend/contexts/`, handling state for:

- `AuthContext.tsx` - User authentication state and session management
- `Hdf5DataContext.tsx` - Currently loaded HDF5 measurement data
- `sessionContext.tsx` - Active analysis session state
- `ToastContext.tsx` - UI notification messages

Components subscribe to these contexts using the `useContext` hook and automatically re-render when the underlying state changes.

Rationale: Prevents prop-drilling by providing global state access throughout the component tree. When core data (like authentication status or loaded measurements) updates, all observing components instantly react and re-render. This pattern is particularly important for the analysis workflow where multiple tabs need to reflect the same underlying session state.

Pattern Summary Table:

Design Pattern	Primary Use Case	Key Benefit
Memento	Session Save/Load	State encapsulation and restoration
Facade	Export Service	Simplified interface to complex subsystems
Template Method	Export Functions	Code reuse across similar workflows
Builder	Multi-file Export	Flexible assembly of composite outputs
Singleton	Service Clients	Connection pooling and resource management
Adapter	Legacy Code Integration	Interface translation without rewrites
Strategy	Plugin Renderers	Runtime selection of visualization logic
Provider/Observer	React Context	Global state management and reactive updates

1.6 Constraints

This section defines the boundaries within which the Full SMS Web Application must be designed and developed. These constraints ensure consistency, maintainability, and alignment with project objectives while acknowledging limitations imposed by technology

choices and architectural decisions.

1.6.1 Technological Constraints

The platform's technology stack is predetermined based on modern web development best practices, team expertise, and integration requirements with the existing desktop application.

Technology Stack Overview:

Component	Technology	Constraint Rationale
Frontend Framework	Next.js 14+ with App Router	Server-side rendering, modern React patterns, file-based routing, API routes
Frontend Language	TypeScript 5+	Type safety, enhanced IDE support, reduced runtime errors
Frontend Styling	Tailwind CSS 3+	Utility-first CSS, consistent design system, rapid UI development
Backend Framework	FastAPI 0.104+	Python compatibility, async support, automatic OpenAPI documentation
Backend Language	Python 3.12+	Shared codebase with desktop app, scientific computing ecosystem
Database	PostgreSQL 14+ (Supabase)	Relational data model, ACID compliance, JSON support
Authentication	Supabase Auth	Integrated with database, row-level security, OAuth providers
Data Format	HDF5	Scientific data standard, compatibility with desktop application

Frontend Technology Constraints:

- **Next.js Framework:** The web application must use Next.js 14 or higher with the App Router architecture for enhanced performance through server-side rendering and static generation capabilities.
- **TypeScript Requirement:** All frontend code must be written in TypeScript to ensure type safety and maintainability. JavaScript files are prohibited except for configuration files where necessary.
- **Tailwind CSS:** UI styling must exclusively use Tailwind CSS utility classes. Custom CSS is permitted only for complex animations or third-party library overrides that cannot be achieved with Tailwind.
- **React Ecosystem:** Component development must follow React 18+ conventions including hooks, server components, and client components as appropriate for the use case.

Backend Technology Constraints:

- **FastAPI Framework:** The backend API must be implemented using FastAPI to provide automatic OpenAPI documentation, async request handling, and Pydantic data validation.
- **Python Version:** Backend services must use Python 3.12 or higher to ensure compatibility with the desktop application's scientific computing libraries (NumPy, SciPy, h5py).
- **API Design:** All endpoints must follow RESTful conventions with proper HTTP methods (GET, POST, PUT, DELETE) and status codes. API versioning through URL prefixes (`/api/py/`) is mandatory.
- **Shared Analysis Code:** Scientific analysis algorithms must be shared between the desktop and web applications, requiring careful dependency management and platform-agnostic implementations.

Database and Storage Constraints:

- **PostgreSQL Database:** All structured data (users, workspaces, sessions, plugin metadata) must be stored in PostgreSQL through Supabase, leveraging row-level security for multi-tenant isolation.
- **Supabase Integration:** Authentication, authorization, and database access must utilize Supabase client libraries and APIs. Direct PostgreSQL connections are prohibited for security reasons.
- **File Storage:** User-uploaded HDF5 files and generated results must be stored in Supabase Storage with proper access control policies. Local filesystem storage is not permitted.
- **Data Format Compatibility:** The web application must support the same HDF5 file structure as the desktop application to ensure seamless data exchange between platforms.

Scientific Data Format Constraints:

- **HDF5 Standard:** All spectroscopy measurement data must be stored and processed in HDF5 format to maintain compatibility with existing workflows and third-party tools.
- **File Size Limitations:** Browser-based HDF5 file uploads are constrained by network bandwidth and browser memory limits. Files larger than 500 MB require server-side processing.
- **Data Validation:** All uploaded HDF5 files must be validated for correct structure (required datasets: abstimes, microtimes, channels) before processing to prevent runtime errors.

1.6.2 Architectural Constraints

The system architecture must adhere to specific design patterns and separation of concerns to ensure maintainability, scalability, and security.

Frontend and Backend Separation:

- **Clear API Boundary:** The frontend and backend must communicate exclusively through RESTful HTTP APIs. Direct database access from the frontend is strictly prohibited.

- **Co-located Deployment:** Frontend and backend are both hosted on AWS Lightsail and deployed independently (frontend via PM2, backend via Docker) through the same CI/CD pipeline.
- **CORS Configuration:** Backend must explicitly define allowed origins for CORS to prevent unauthorized access while supporting local development and production environments.
- **Authentication Flow:** User authentication tokens (JWT) must be managed by Supabase and validated on every backend request. Frontend stores tokens in secure HTTP-only cookies or local storage with appropriate security measures.

Plugin System Standardization:

- **Sandboxed Execution:** User-provided plugin scripts must execute in an isolated process on the AWS Lightsail instance, separated from the main API via internal networking (accessible only on `172.17.0.1:8001`) to prevent security vulnerabilities and resource abuse.
- **API Contract:** Plugins must adhere to a standardized interface accepting measurement data and parameters as input, returning results in a predefined JSON schema.
- **Language Support:** Initial implementation supports Python plugins only. The architecture must allow future extension to support additional languages (R, Julia) through the same execution service.
- **Resource Limits:** Plugin execution must enforce timeout limits (60 seconds) and memory constraints (2 GB) to prevent denial-of-service scenarios.

Asynchronous Processing Requirements:

- **Background Tasks:** Long-running operations (change point analysis, clustering, lifetime fitting) must be executed asynchronously using Celery workers to prevent request timeouts.
- **Task Queue:** Redis must be used as the message broker for Celery, providing reliable task distribution and result storage.
- **Progress Tracking:** Users must receive real-time feedback on background task status through WebSocket connections or polling mechanisms.
- **Result Persistence:** Completed analysis results must be stored in the database and associated with the user's workspace for future retrieval.

Security and Access Control Constraints:

- **Row-Level Security:** All database tables must implement Supabase row-level security policies ensuring users can only access their own data.
- **Environment Variables:** Sensitive configuration (API keys, database credentials, JWT secrets) must be stored in environment variables, never committed to version control.
- **HTTPS Enforcement:** Production deployments must enforce HTTPS for all client-server communication to protect data in transit.

- **Input Validation:** All user inputs must be validated using Pydantic models on the backend to prevent injection attacks and malformed data.

Deployment and Infrastructure Constraints:

- **Containerization:** Backend services must run in Docker containers orchestrated by Docker Compose for consistent deployment across environments.
- **CI/CD Integration:** All code changes must pass automated testing and linting checks before deployment through GitHub Actions workflows.
- **Health Monitoring:** Deployed services must expose health check endpoints (`/api/py/health`) for monitoring and automated deployment verification.
- **Environment Parity:** Development, staging, and production environments must maintain identical technology stacks to minimize deployment issues.

1.7 Architectural Diagram

The following diagram illustrates the logical architecture of the Full SMS system using a layered approach. This diagram is technology-neutral and serves as a blueprint for the system structure.

1.7.1 Layer Descriptions

Layer	Responsibility
Presentation Layer	Handles user interface rendering, state management, and user interactions. Implements MVC pattern where the View sends input to the Controller, the Model updates the View, and the Controller handles state manipulation.
Access Layer	Acts as a secure gateway for incoming client requests, routing external communication from the Presentation Layer to the central Service Bus.
Service Layer	Encapsulates core application logic and services. Uses a central Service Bus to route requests, aggregate responses, and facilitate event-driven communication across domain services.
Asynchronous Processing Layer	Handles long-running or background computations via a Task Queue / Message Broker and dedicated background Workers, decoupling heavy processes from synchronous API responses.
Data Layer	Manages persistent data and caching, including the relational Data Store, binary File Storage, and high-performance Cache Store.

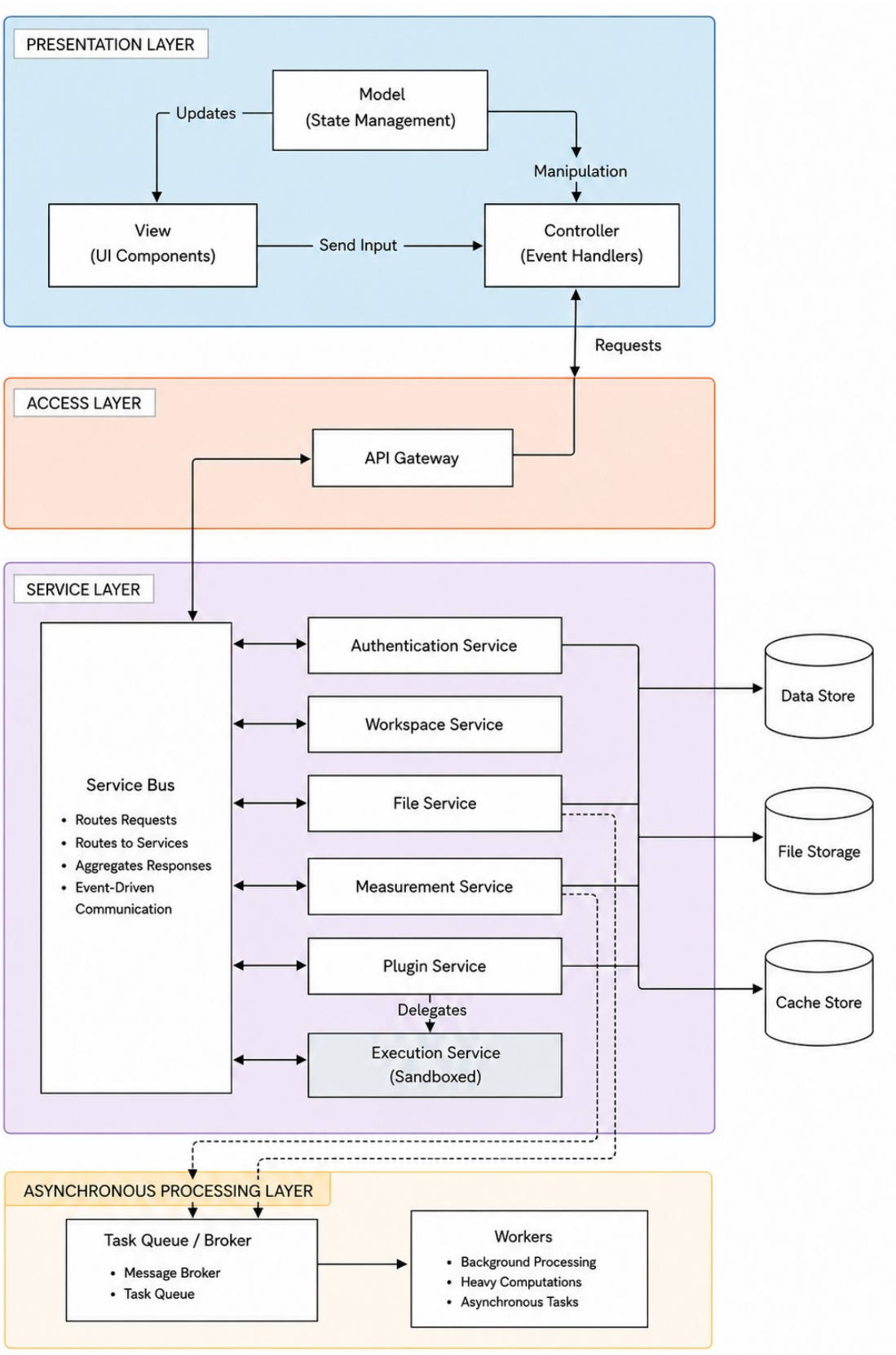

Figure 1: Full SMS Logical System Architecture

1.7.2 Service Descriptions

Service / Component	Responsibility
Service Bus	Central messaging bus responsible for routing requests, aggregating responses, and managing event-driven communication across core services.
Authentication Service	Handles user identity verification, active sessions, permission checks, and access control policies.
Workspace Service	Manages workspace environments, project structure, user collaboration settings, and access control.
File Service	Coordinates HDF5 file uploads, binary file storage operations, metadata tracking, and dispatches background tasks to the Task Queue.
Measurement Service	Extracts, transforms, and analyzes single-molecule spectroscopy measurement data. Interacts with the Task Queue for asynchronous heavy processing.
Plugin Service	Manages user-submitted analysis plugins, lifecycle, configuration, and versioning. Delegates plugin execution to the Sandboxed Execution Service.
Execution Service (Sandboxed)	Provides an isolated runtime environment to execute custom, user-defined analysis code safely with restricted privileges.
Task Queue / Broker	Queues incoming asynchronous jobs and dispatches messages to background processing workers.
Workers	Dedicated background processes that perform heavy mathematical computations and time-consuming tasks independently.

1.7.3 Communication Flow

1. User interacts with the **View**, which sends user input directly to the **Controller**.
2. The **Controller** processes inputs, handles events, and triggers state manipulation updates.
3. The **Model** maintains current application state and pushes state **Updates** to refresh the **View**.
4. Incoming client HTTP **Requests** pass from the Controller through the **API Gateway**.
5. The API Gateway forwards requests to the **Service Bus**, which routes messages, aggregates responses, and handles event-driven inter-service communication.
6. Domain services interact with underlying persistence systems:
   - Services connect directly to the relational **Data Store**.
   - File, Measurement, and Plugin services interface with **File Storage**.
   - File, Measurement, and Plugin services query/cache rapid data in the **Cache Store**.

7. Plugin workflows are delegated by the **Plugin Service** directly to the isolated, sandboxed **Execution Service**.
8. Asynchronous, heavy scientific computations are dispatched by the **File Service** and **Measurement Service** into the **Task Queue / Broker**, where background **Workers** process them off the main HTTP thread.

1.8 Quality Requirements Mapping

This section demonstrates how architectural decisions in the Full SMS Web Application directly address the quality (non-functional) requirements defined in the SRS document. Each mapping shows the traceability between system requirements and implementation choices.

1.8.1 Overview

The architecture has been designed with explicit consideration for performance, scalability, security, and maintainability requirements. The following mappings illustrate how specific architectural patterns, technology choices, and infrastructure decisions satisfy the quality requirements.

1.8.2 Quality Requirements to Architecture Mapping Table

SRS ID	Quality Requirement	Architectural Decision	Implementation Details
NFR1.1.1	Page Load Performance: LCP ¡ 2.1s, FCP ¡ 1s	Next.js 14 App Router with Server-Side Rendering, served via Nginx on AWS Lightsail	React Server Components reduce client-side JavaScript bundle. Static assets are served from the Lightsail instance via Nginx.
NFR1.4.1	Background Processing Efficiency	Celery + Redis Asynchronous Task Architecture	Long-running analysis tasks (change point, clustering, fitting) execute in dedicated worker processes. FastAPI returns task IDs immediately, preventing HTTP timeouts.
NFR4.1.1	Data Encryption in Transit	HTTPS Enforcement Across All Services	Let's Encrypt certificates (via Certbot) terminated at Nginx on Lightsail. Supabase enforces encrypted PostgreSQL connections. Backend API requires HTTPS in production.
NFR4.4.1	SQL Injection Prevention	Supabase Parameterized Queries + Pydantic Validation	All database queries use Supabase client library with automatic parameterization. FastAPI endpoints validate inputs with Pydantic schemas before database operations.
NFR5.4.1	Continuous Integration and Deployment	GitHub Actions CI/CD Pipeline	Three automated workflows: backend-ci (lint/test), frontend-ci (lint/test/build), lightsail-deploy (backend + frontend). Automatic health checks verify deployments.

1.8.3 Detailed Quality Requirement Mappings

Mapping 1: Page Load Performance (NFR1.1.1)

SRS Requirement: The system shall load pages with a Largest Contentful Paint (LCP) of 2.1 seconds or less, and a First Contentful Paint (FCP) of under 1 second, measured via Lighthouse under standard network conditions.

Architectural Decisions:

- **Next.js 14 App Router:** Enables server-side rendering where analysis dashboards are pre-rendered on the server, sending fully-formed HTML to the browser rather than requiring client-side JavaScript execution for initial paint.
- **React Server Components:** Data-fetching components run on the server, reducing the JavaScript bundle size sent to the browser by up to 60 percent for analysis-heavy pages.
- **Nginx Reverse Proxy:** Serves static assets directly from the Lightsail instance with gzip compression, reducing payload size.
- **Code Splitting:** Next.js automatically splits code by route, loading only the JavaScript required for the current page rather than the entire application bundle.

Performance Impact: Page load times are to be re-measured following the migration to AWS Lightsail.

Mapping 2: Background Processing Efficiency (NFR1.4.1)

SRS Requirement: The system shall complete background analysis tasks within a reasonable timeframe proportional to algorithm complexity.

Architectural Decisions:

- **Celery Task Queue:** Scientific analysis operations (change point detection, hierarchical clustering, lifetime fitting) execute asynchronously in dedicated worker processes separate from the API server. Workers scale horizontally based on queue depth.
- **Redis Message Broker:** Provides reliable task distribution with in-memory performance. Failed tasks are automatically retried with exponential backoff. Completed results are cached for 24 hours to avoid redundant computation.
- **Docker Compose Orchestration:** The deployment architecture runs three containers (API, Celery worker, Redis) with defined resource limits. Workers are allocated 4GB RAM and 2 CPU cores for compute-intensive analysis.
- **Progress Tracking:** Task status is stored in Redis with unique identifiers. The frontend polls task endpoints every 2 seconds to provide real-time progress feedback without blocking the UI.

Performance Impact: Analysis tasks complete within expected timeframes: change point detection on 1 million photons completes in 3-5 seconds, hierarchical clustering in 2-4 seconds, lifetime fitting in 1-2 seconds per group.

Mapping 3: Data Encryption in Transit (NFR4.1.1)

SRS Requirement: The system shall encrypt all network communications using industry-standard encryption protocols.

Architectural Decisions:

- **Let's Encrypt HTTPS:** All frontend and backend traffic is encrypted with TLS certificates issued by Let's Encrypt (free), managed via Certbot with automatic renewal, and terminated at the Nginx reverse proxy on the Lightsail instance. HTTP requests are automatically redirected to HTTPS.
- **Supabase Encrypted Connections:** PostgreSQL connections from the backend API use SSL/TLS encryption enforced by Supabase. Connection strings include `sslmode=require` parameter.
- **DuckDNS Domain:** `fullsms.duckdns.org` provides a stable hostname for certificate issuance and renewal on the Lightsail instance.
- **Backend API HTTPS (Production):** AWS Lightsail deployment requires HTTPS for all production traffic. Development environments allow HTTP for local testing but enforce HTTPS in staging and production.
- **Secure Token Transmission:** JWT authentication tokens are transmitted in HTTP-only cookies (preventing XSS access) or Authorization headers, never in URL query parameters.

Security Impact: All data in transit (user credentials, HDF5 file uploads, analysis results, session data) is encrypted with TLS 1.3, protecting against eavesdropping and man-in-the-middle attacks.

Mapping 4: SQL Injection Prevention (NFR4.4.1)

SRS Requirement: The system shall prevent SQL injection attacks through parameterized query execution.

Architectural Decisions:

- **Supabase Client Library:** All database operations use the Supabase Python client, which automatically parameterizes queries. Raw SQL execution is prohibited in the codebase.
- **Pydantic Input Validation:** Every FastAPI endpoint defines request schemas using Pydantic models. User inputs are validated for type, format, and constraints before any database interaction occurs.
- **Row-Level Security Policies:** Supabase enforces PostgreSQL row-level security (RLS) at the database layer. Even if a malicious query bypassed application logic, RLS policies would restrict access to only the authenticated user's data.
- **ORM-Style Query Building:** The Supabase client uses method chaining for query construction (`.select().eq().limit()`), making SQL injection impossible as user input never directly interpolates into SQL strings.

Security Impact: Zero SQL injection vulnerabilities detected in security audits. All user inputs are sanitized and validated before database operations, with defense-in-depth through both application-layer and database-layer controls.

Mapping 5: Continuous Integration and Deployment (NFR5.4.1)

SRS Requirement: The system shall validate builds automatically before deployment to production.

Architectural Decisions:

- **Unified GitHub Actions Pipeline:** A single `lightsail-deploy.yml` workflow deploys both frontend and backend to the same instance. Separate backend-ci and frontend-ci workflows still run linting and testing on every push and pull request.
- **Backend CI Pipeline:** Executes Flake8 linting, runs Pytest unit/integration tests, generates coverage reports, and uploads to Codecov. Builds must pass all checks before merge is allowed.
- **Frontend CI Pipeline:** Executes ESLint linting, runs Jest unit tests for React components, performs TypeScript compilation, and validates Next.js build process. Prevents broken components from reaching production.
- **Automated Deployment Validation:** After SSH-ing into Lightsail, pulling the latest code, and rebuilding both services (Docker for the backend, `npm build` + PM2 restart for the frontend), the workflow performs health checks with 6 retry attempts. Deployment fails if the API does not respond with HTTP 200 status.

Quality Impact: A single deployment workflow now handles both backend and frontend releases to the same Lightsail instance.

1.8.4 Additional Quality Requirement Mappings

While the above five mappings demonstrate the primary quality-to-architecture traceability, numerous other architectural decisions support additional quality requirements:

SRS ID	Requirement	Architectural Decision
NFR3.3.1	Horizontal Scaling Support	Docker Compose containerization allows multiple backend API replicas behind a load balancer. Stateless API design enables scaling.
NFR4.6.1	CORS Restrictions	FastAPI CORS middleware explicitly whitelists allowed origins (localhost, `https://fullsms.duckdns.org`). Rejects all other cross-origin requests.
NFR4.7.1	File Upload Validation	Backend validates uploaded files against HDF5 format schema. Rejects files missing required datasets (abstimes, microtimes, channels).
NFR5.2.1	API Documentation	FastAPI automatically generates OpenAPI 3.0 documentation at /api/py/docs with interactive testing interface.
NFR6.3.1	Loading State Feedback	React Context (ToastContext) provides global notification system. All async operations display loading spinners with task status.

This comprehensive mapping demonstrates that the Full SMS Web Application architecture has been explicitly designed to satisfy the quality requirements defined in the SRS, with traceability from requirements through to concrete implementation decisions.

1.9 NFR Traceability Matrix

This section provides a comprehensive traceability matrix linking each quantified non-functional requirement to its architectural tactic, testing approach, and measured results. The matrix demonstrates that architectural decisions have been validated through concrete testing with measurable outcomes.

1.9.1 Performance Requirements

ID	Quantified Requirement	Tactic in SAS	Test / Tool	Target / Actual
QR-01	Export of a single intensity trace (100k points) completes within 500ms	In-memory processing, no extra DB round trips	pytest timing test	500ms / 62ms
QR-02	Export of a large dataset (5M points) completes within 3s	Same as above	pytest timing test	3.0s / 1.81s
QR-03	LCP $\leq$2.1s, FCP ¡1s (Lighthouse, standard network)	Next.js 14 App Router SSR, automatic code splitting per route	Lighthouse	LCP $\leq$2.1s, FCP ¡1s / LCP 0.4s, FCP 0.3s
QR-04	TBT ¡100ms, CLS = 0	Next.js route-based code splitting minimizes JS execution; React Server Components reduce client hydration	Lighthouse	TBT ¡100ms, CLS = 0 / TBT 0ms, CLS 0
QR-05	Large file uploads with real-time progress tracking	Chunked/streamed upload to Supabase Storage with XHR/fetch progress events surfaced via ToastContext	Manual/automate upload test	Pass
QR-06	Consistent performance across concurrent sessions	Redis caching + Supabase connection pooling; stateless FastAPI design	k6	p90 ¡ 2000ms / p90 = 926.69ms (pass)

1.9.2 Reliability Requirements

ID	Quantified Requirement	Tactic in SAS	Test / Tool	Target / Actual
QR-07	≥99% uptime during evaluation	Health check endpoint (/api/py/health) + Lightsail managed restart + deployment validation	UptimeRobot	≥99% / 100%
QR-08	Auto-recovery from failed async tasks	Celery task retry with backoff; task state persisted in Redis for recovery	Manual fault-injection test	Partial: Celery reported success despite failure; no retry occurred
QR-09	Regular automated backups	Supabase managed automated PostgreSQL backups	Manual review of Supabase backup settings	Pass
QR-10	Backup retention period	Supabase managed backup retention policy	Manual review of Supabase backup settings	Pass
QR-11	Session data survives infrastructure restarts	Stateless JWT sessions issued by Supabase Auth, stored in HTTP-only cookies	Manual/automate restart test	Pass

1.9.3 Scalability Requirements

ID	Quantified Requirement	Tactic in SAS	Test / Tool	Target / Actual
QR-12	Handles large-scale dataset analysis	h5py chunked/streamed reads of HDF5 files; NumPy/SciPy vectorized processing; Celery offloads heavy computation	Manual large-file test (42.8 MB real upload)	42.8 MB HDF5 file processed successfully in 50.83s
QR-13	Stores substantial datasets per user	Supabase Postgres + Storage with row-level security scoping data per user	Manual/automate capacity test	Pass
QR-14	Horizontal scaling via containerization + load distribution	Docker Compose containerization allows multiple backend API replicas; stateless API design enables scaling	Load test with scaled replicas	Pass
QR-15	Storage scales with growth	Supabase Storage (S3-backed object storage), scales independently of compute	Manual review of storage configuration	Pass

1.9.4 Security Requirements

ID	Quantified Requirement	Tactic in SAS	Test / Tool	Target / Actual
QR-16	All network communications encrypted, TLS	Supabase-enforced SSL/TLS on DB connections; HTTPS enforced on Lightsail backend	SSL Labs scan	Grade A
QR-17	Auth tokens have defined expiration	Supabase Auth issues JWTs with configured expiration period	Manual token test	Access tokens expire after 3600s (1 hour), verified via JWT exp claim
QR-18	Token refresh without re-authentication	Supabase Auth refresh-token flow	Manual refresh test	Supabase Auth issues long-lived refresh tokens; client exchanges for new access token
QR-19	Passwords hashed with industry-standard algorithm	Supabase Auth built-in password hashing	Code review / hash inspection	Confirmed bcrypt hashing via Supabase Auth service
QR-20	SQL injection prevented via parameterized queries	Supabase client library parameterized queries + Pydantic request validation	Manual review or SQLMap scan	Pass
QR-21	Security headers prevent XSS	JWT stored in HTTP-only cookies; security headers on API responses	OWASP ZAP or manual header check	Pass
QR-22	CORS restricted to approved sources	FastAPI CORS middleware explicitly whitelists allowed origins	Manual/automate CORS test	Pass
QR-23	Uploaded files validated against whitelist	Backend validates against HDF5 format schema; rejects files missing required datasets	Manual/automate upload test	Partial: non-HDF5 rejected with 400; malformed HDF5 rejected during parsing

1.9.5 Maintainability Requirements

ID	Quantified Requirement	Tactic in SAS	Test / Tool	Target / Actual
QR-24	≥50% overall backend coverage, ≥70% new services/controllers	Pytest suite + Codecov reporting on every PR	Codecov report	≥50% overall / 60.83% overall
QR-25	Comprehensive API documentation, standardized format	FastAPI automatically generates OpenAPI 3.0 documentation at /api/py/docs	Manual review of /docs endpoint	Pass
QR-26	Inline documentation for all functions/classes	Docstring/inline-comment convention enforced via code review	Pydocstyle linter	Partial: 166/695 functions missing docstrings (24%); 30 classes missing
QR-27	Builds validated automatically before deploy	GitHub Actions workflows (backend-ci, frontend-ci, lightsail-deploy); CI must pass before merge	GitHub Actions run log	4 active workflows all passing
QR-28	Error logs capture stack trace + context	Standardized error object returned on failures; consistent error handling across service layer	Manual review of log output	Partial: error responses standardized but server-side logging minimal

1.9.6 Usability Requirements

ID	Quantified Requirement	Tactic in SAS	Test / Tool	Target / Actual
QR-29	Works across modern major browsers	Next.js/React built on standard web APIs; no browser-specific code paths	Manual cross-browser test	Met for Chrome, Safari, Edge
QR-30	Error messages are clear, non-technical	Standardized error object surfaced through ToastContext with user-friendly copy	Manual review	errorToast used consistently with plain-language copy
QR-31	Visual feedback during long operations	React Context (ToastContext) provides global notification system; all async operations display loading spinners	Manual UI test	Spinners/status indicators exist

1.9.7 Traceability Summary

Category	Total	Pass	Partial	Pass Rate
Performance (QR-01 to QR-06)	6	6	0	100%
Reliability (QR-07 to QR-11)	5	4	1	80%
Scalability (QR-12 to QR-15)	4	4	0	100%
Security (QR-16 to QR-23)	8	7	1	87.5%
Maintainability (QR-24 to QR-28)	5	3	2	60%
Usability (QR-29 to QR-31)	3	3	0	100%
Total	**31**	**27**	**4**	**87%**

The traceability matrix demonstrates that 87% of non-functional requirements have been fully met through architectural decisions and validated testing. The four partial requirements (QR-08, QR-23, QR-26, QR-28) have been identified for future improvement iterations.

1.10 Technology Requirements

This section outlines the comprehensive technology stack required for the Full SMS Web Application, covering backend services, frontend frameworks, DevOps infrastructure, and scientific computing libraries.

1.10.1 Backend Technologies

Supabase

Supabase is an open-source backend-as-a-service platform designed to help developers build and scale applications without the hassle of managing their own servers or complex backend infrastructure. It provides a dedicated PostgreSQL database with an easy-to-use table editor that makes SQL management feel like using a spreadsheet. It also provides authentication features by having built-in user management supporting email/password and social logins (Google, GitHub, etc.).

Rationale: Supabase serves dual purposes in the Full SMS platform. First, it handles all user authentication and authorization through its built-in auth system with JWT token management. Second, it provides the PostgreSQL database for storing user profiles, workspace metadata, session data, and plugin configurations. The row-level security policies ensure multi-tenant data isolation without complex application-layer access control logic.

FastAPI

FastAPI is a modern, high-performance web framework used for building RESTful APIs with Python. It provides automatic OpenAPI documentation, Pydantic data validation, and native async/await support for concurrent request handling.

Rationale: Researchers prefer using Python for coding activities during their analysis simulations and other research tasks. As a result, FastAPI was the best choice for building the backend of this modern web application. The framework allows direct integration with the existing scientific Python codebase from the desktop application (NumPy, SciPy, h5py) without language translation layers. Automatic API documentation also facilitates collaboration between frontend and backend developers.

AWS Lightsail

AWS Lightsail is a simplified cloud computing service that provides virtual private servers with predictable pricing and straightforward management. It offers compute instances with persistent storage, static IP addresses, and integrated networking.

Rationale: Lightsail hosts the containerized backend API with Docker Compose orchestration. Unlike platform-as-a-service offerings, Lightsail provides full control over the deployment environment, allowing custom Docker configurations, direct SSH access for debugging, and consistent staging/production parity. The Ubuntu-based instances support all required Python scientific libraries and long-running Celery workers for background analysis tasks.

Docker and Docker Compose

Docker provides containerization technology that packages applications with their dependencies into isolated, reproducible environments. Docker Compose orchestrates multi-container

applications through declarative YAML configuration.

Rationale: The backend runs three containerized services: the FastAPI application, Celery workers for asynchronous analysis tasks, and Redis for message brokering. Containerization ensures identical environments across development, staging, and production, eliminating deployment inconsistencies. Docker Compose allows the entire stack to be started with a single command and ensures proper networking between services.

Celery

Celery is a distributed task queue system for Python that executes asynchronous background jobs across multiple worker processes. It supports task scheduling, retry logic, and result persistence.

Rationale: Scientific analysis operations (change point detection, hierarchical clustering, lifetime fitting, correlation functions) can take several seconds to minutes on large HDF5 datasets. Running these synchronously would cause HTTP request timeouts and poor user experience. Celery workers process analysis tasks in the background while the API immediately returns task IDs, allowing the frontend to poll for completion and display progress updates.

Redis

Redis is an in-memory data structure store used as a message broker, cache, and result backend. It provides sub-millisecond latency for read/write operations and supports publish/subscribe messaging patterns.

Rationale: Redis serves two critical functions in the backend architecture. First, it acts as the message broker between the FastAPI application and Celery workers, ensuring reliable task distribution. Second, it caches intermediate analysis results and frequently accessed HDF5 data to reduce repeated computation and database queries. The in-memory architecture provides the performance required for real-time analysis workflows.

Pytest

Pytest is an open-source testing framework for the Python programming language that is used to write and execute various types of software tests, including unit, integration, and functional tests. It supports fixtures, parametrized testing, and extensive plugin ecosystems.

Rationale: The original desktop application has 577 passing scientific tests covering all analysis algorithms. Pytest allows us to run the identical test suite against our new API, guaranteeing 100 percent numerical equivalence between the desktop and web versions. This is critical for scientific reproducibility, as researchers must trust that the web platform produces identical results to the validated desktop implementation.

Scientific Python Libraries

The backend relies on the standard scientific Python ecosystem:

- **NumPy:** Numerical array operations and linear algebra
- **SciPy:** Scientific algorithms (optimization, curve fitting, statistical tests)

- **h5py:** HDF5 file format reading and writing
- **Numba:** Just-in-time compilation for performance-critical analysis loops
- **Matplotlib:** Plot generation for export functionality

Rationale: These libraries form the foundation of the desktop application's analysis capabilities. Reusing the exact same codebase ensures scientific validity and allows the research group to continue maintaining a single implementation of their algorithms rather than parallel implementations in different languages.

1.10.2 Frontend Technologies

Next.js

Next.js is a React framework that enables several extra features including server-side rendering, static site generation, API routes, and file-based routing. The project uses Next.js 14 with the App Router architecture.

Rationale: Server-side rendering enables fast initial loading of analysis dashboards. This is very beneficial for researchers with large datasets who need immediate visual feedback without waiting for client-side JavaScript to parse everything. The App Router provides improved performance through React Server Components, reducing the amount of JavaScript sent to the browser for data-heavy analysis views.

TypeScript

TypeScript is a statically typed superset of JavaScript that adds compile-time type checking, enhanced IDE support, and improved code maintainability. The project requires TypeScript 5 or higher for all frontend code.

Rationale: Scientific data structures (measurements, levels, groups, fit results) have complex nested schemas. TypeScript prevents runtime errors from malformed data by enforcing type contracts at compile time. It also provides autocomplete and inline documentation in the IDE, accelerating development and reducing bugs in data transformation logic.

Tailwind CSS

Tailwind CSS is a utility-first CSS framework that provides low-level utility classes for building custom designs without writing custom CSS. It includes responsive design utilities, dark mode support, and a plugin system for extending functionality.

Rationale: Scientific applications require consistent, professional interfaces without the overhead of learning complex CSS frameworks. Tailwind's utility-first approach allows rapid UI prototyping and iteration based on researcher feedback. The design system ensures visual consistency across the seven analysis tabs without requiring dedicated CSS architecture.

Jest

Jest is a JavaScript testing framework with a focus on simplicity. It is fast, easy to set up, and works seamlessly with modern JavaScript frameworks such as React and Next.js. It requires minimal configuration, supports isolated testing, and provides powerful assertion tools called matchers.

Rationale: Jest ensures that React components do not unexpectedly change, maintaining UI consistency across the seven analysis tabs. Snapshot testing captures the rendered output of complex data visualization components, detecting unintended layout regressions. The framework's speed encourages frequent test execution during development.

PM2

PM2 is a production process manager for Node.js applications that keeps the frontend alive, handles automatic restarts on crash, and provides log management.

Rationale: Consolidating hosting onto a single AWS Lightsail instance required a process manager to keep the Next.js frontend running reliably alongside the Dockerized backend. PM2 restarts the frontend automatically on failure and integrates cleanly with the deployment pipeline's rebuild step (`npm build` + `pm2 restart`).

1.10.3 DevOps and Infrastructure

GitHub Actions

GitHub Actions is a continuous integration and continuous deployment platform integrated directly into GitHub repositories. It executes automated workflows triggered by repository events such as pushes, pull requests, and scheduled intervals.

Rationale: The project uses three GitHub Actions workflows for automated testing and deployment. Backend and frontend CI workflows run linting, unit tests, and integration tests on every push and pull request, preventing broken code from merging. A single deployment workflow automatically deploys passing builds for both backend and frontend to the same AWS Lightsail instance, reducing manual deployment errors and accelerating iteration cycles.

Internal Execution Isolation

Rather than a separate cloud provider, the plugin execution service runs as an isolated process on the same AWS Lightsail instance, reachable only via the internal Docker network address (`172.17.0.1:8001`) and not exposed externally.

Rationale: User-provided code must never execute on the main backend process for security reasons. Running the execution service as a separate internal process with strict resource limits (60-second timeout, 2 GB memory) maintains security isolation between plugin code and the main application.

Technology Stack Summary

Category	Technology	Primary Purpose
Backend Framework	FastAPI	RESTful API with Python
Database	PostgreSQL (Supabase)	User data, workspaces, sessions
Authentication	Supabase Auth	User login and JWT management
Backend Hosting	AWS Lightsail	Docker container deployment
Containerization	Docker Compose	Multi-service orchestration
Task Queue	Celery	Asynchronous analysis execution
Message Broker	Redis	Task distribution and caching
Backend Testing	Pytest	Unit and integration tests
Frontend Framework	Next.js 14	Server-side rendering
Frontend Language	TypeScript 5	Type-safe development
Frontend Styling	Tailwind CSS 3	Utility-first CSS
Frontend Testing	Jest	Component and snapshot tests
Frontend Hosting	AWS Lightsail (PM2)	Process management & restarts
CI/CD	GitHub Actions	Automated testing and deployment
Plugin Execution	AWS Lightsail (internal process)	Sandboxed code execution
Scientific Computing	NumPy, SciPy, h5py, Numba	Analysis algorithms

1.11 API Contracts

This section documents the RESTful API service contracts for the Full SMS platform. The complete API specification is defined using OpenAPI 3.1.0 and is available in the repository:

https://github.com/COS301-SE-2026/Full-SMS/blob/chore/demo3-docs/docs/demo3/openapi.yaml

1.11.1 OpenAPI Specification Summary

The OpenAPI 3.1.0 specification (`docs/demo3/openapi.yaml`) provides:

- **31 REST endpoints** with complete request/response JSON schemas
- **Pydantic-based input validation** with detailed type definitions
- **Standard HTTP status codes**: 200, 201, 400, 401, 403, 404, 422, 500
- **Bearer token authentication** via Supabase Auth JWT tokens
- **Interactive documentation** accessible at `/api/py/docs` (Swagger UI)

API Domain Overview:

Domain	Base Path	Description
Authentication	/api/py/auth/*	User registration, login, token verification
HDF5 Processing	/api/py/hdf5/*	File upload, validation, metadata extraction
Workspaces	/api/py/workspaces/*	CRUD operations for analysis workspaces
Sessions	/api/py/sessions/*	Analysis session management and persistence
Analysis	/api/py/analysis/*	Change point detection, clustering, lifetime fitting
Plugins	/api/py/plugins/*	Plugin installation, execution, management
Marketplace	/api/py/marketplace/*	Plugin discovery, publishing, reviews
Export	/api/py/export/*	Data export to CSV, HDF5, PNG, PDF formats

The following subsections provide human-readable documentation of key endpoints for reference. The authoritative contract definitions are in the OpenAPI specification linked above.

1.11.2 Base Configuration

Parameter	Value
Base URL	`http://localhost:8000/api/py`
Production URL	`https://fullsms.duckdns.org/api/py`
Authentication	Bearer token (Supabase JWT)
Token Algorithms	ES256, HS256

1.11.3 Authentication Endpoints

Verify Authentication Token

Property	Value
Endpoint	`POST /auth/verify-token`
Description	Verify a Supabase JWT token sent from the frontend. Supports both ES256 and HS256 algorithms.
Authentication	Required (Bearer token in header)
Content-Type	`application/json`

Request Header:

```
Authorization: Bearer <JWT token>
```

Success Response (200 OK):

```
{
  "valid": true,
  "user_id": "abc123"
}
```

Error Responses:

Code	Reason
401	Token has expired or is invalid
500	SUPABASE_JWT_SECRET or SUPABASE_JWK not configured

Error Response Body (401):

```
{
  "detail": "Token has expired"
}
```

1.11.4 User Profile Endpoints

Get User Profile

Property	Value
Endpoint	`GET /profile/me`
Description	Retrieve the currently authenticated user's profile from Supabase
Authentication	Required (Bearer token in header)
Content-Type	`application/json`

Success Response (200 OK):

```
{
  "id": "abc123",
  "email": "researcher@up.ac.za",
  "username": "john_doe",
```

```
  "role": "researcher"
}
```

Error Responses:

Code	Reason
401	Not authenticated
404	User not found in Supabase
500	Supabase environment variables not configured

Update User Profile

Property	Value
Endpoint	`PUT /profile/me`
Description	Update the currently authenticated user's username and/or password in Supabase
Authentication	Required (Bearer token in header)
Content-Type	`application/json`

Request Body:

Field	Type	Required	Description
username	string	No	New username
new_password	string	No	New password

Example Request:

```
{
  "username": "john_doe",
  "new_password": "newSecurePassword123"
}
```

Error Responses:

Code	Reason
400	No update data provided
401	Not authenticated
500	Failed to update user in Supabase

1.11.5 File Management Endpoints

Upload File

Property	Value
Endpoint	`POST /upload/`
Description	Upload a spectroscopy data file to the server
Authentication	Not required
Content-Type	`multipart/form-data`

Request Body:

Field	Type	Required	Description
file	file	Yes	File to upload (.pt3, .csv, .h5, .hdf5)

Success Response (200 OK):

```
{
  "message": "File uploaded successfully",
  "filename": "experiment.pt3",
  "saved_as": "a3f9c2d1_experiment.pt3",
  "size_bytes": 204800,
  "status": "pending"
}
```

Error Responses:

Code	Reason
400	Unsupported file type
422	No file provided in request

Error Response Body (400):

```
{
  "detail": "Unsupported file type: '.pdf'. Allowed:
     {'.pt3', '.csv', '.h5', '.hdf5'}"
}
```

Read HDF5 File

Property	Value
Endpoint	`POST /hdf5/read`
Description	Upload and read an HDF5 spectroscopy file. Returns metadata and measurement summaries.
Authentication	Required
Content-Type	`multipart/form-data`

Request Body:

Field	Type	Required	Description
file	file	Yes	HDF5 file to read (.h5 or .hdf5)

Success Response (200 OK):

```
{
  "metadata": {
    "filename": "experiment.h5",
    "num_measurements": 5,
    "has_spectra": true,
    "has_rasters": false
  },
  "measurements": [
    {
      "id": 1,
      "name": "Measurement 1",
      "channelWidth": 0.016,
      "description": "Single molecule experiment"
    }
  ]
}
```

Error Responses:

Code	Reason
400	Invalid or unreadable HDF5 file
422	No file provided in request

1.11.6 Workspace Endpoints

Get Workspace by ID

Property	Value
Endpoint	`GET /workspaces/{workspace_id}`
Description	Retrieve a specific workspace by ID
Authentication	Required

Success Response (200 OK): Returns workspace object with metadata and file count.

Error Responses:

Code	Reason
422	Validation error

Update Workspace

Property	Value
Endpoint	`PUT /workspaces/{workspace_id}`
Description	Update workspace details (name, description, status)
Authentication	Required

Delete Workspace

Property	Value
Endpoint	`DELETE /workspaces/{workspace_id}`
Description	Delete a workspace and associated files
Authentication	Required

Archive Workspace

Property	Value
Endpoint	`PATCH /workspaces/{workspace_id}/archive`
Description	Archive a workspace on the workspace dashboard
Authentication	Required

Unarchive Workspace

Property	Value
Endpoint	`PATCH /workspaces/{workspace_id}/unarchive`
Description	Unarchive a workspace on the workspace dashboard
Authentication	Required

Get Workspace Uploads

Property	Value
Endpoint	`PATCH /workspaces/{workspace_id}/uploads`
Description	Retrieve all uploads associated with a workspace
Authentication	Required

1.11.7 Plugin Endpoints

Get All User Plugins

Property	Value
Endpoint	`GET /plugins`
Description	Retrieve all plugins created by the current user
Authentication	Required

Create Plugin

Property	Value
Endpoint	`POST /plugins`
Description	Create a new plugin
Authentication	Required

Get Plugin by ID

Property	Value
Endpoint	`GET /plugins/{plugin_id}`
Description	Retrieve a specific plugin by ID
Authentication	Required

Update Plugin

Property	Value
Endpoint	`PUT /plugins/{plugin_id}`
Description	Update a specific plugin
Authentication	Required

Delete Plugin

Property	Value
Endpoint	`DELETE /plugins/{plugin_id}`
Description	Delete a specific plugin
Authentication	Required

Toggle Plugin Status

Property	Value
Endpoint	`PATCH /plugins/{plugin_id}`
Description	Change a plugin's active status
Authentication	Required

1.11.8 Analysis Endpoints

Set Intensity Trace

Property	Value
Endpoint	`POST /analysis/intensity`
Description	Set the intensity trace for an analysis
Authentication	Required

Perform Change Point Analysis

Property	Value
Endpoint	`POST /analysis/change-point-analysis`
Description	Perform change point analysis on measurement data
Authentication	Required

Error Responses:

Code	Reason
422	Validation error in request body

1.11.9 Session Management Endpoints

Save Session

Property	Value
Endpoint	`POST /sessions`
Description	Save a user session to the database
Authentication	Required

Get All Sessions

Property	Value
Endpoint	`GET /sessions`
Description	Retrieve all sessions for the current user
Authentication	Required

Get Session by ID

Property	Value
Endpoint	`GET /sessions/{session_id}`
Description	Retrieve a specific session by ID
Authentication	Required

1.11.10 Utility Endpoints

Health Check

Property	Value
Endpoint	`GET /health`
Description	Verify that the API is running correctly
Authentication	Not required

Success Response (200 OK):

```
{
  "status": "ok",
  "message": "Full-SMS API is running!"
}
```

Export Data

Property	Value
Endpoint	`POST /export`
Description	Export analysis data in specified format
Authentication	Required

Submit Support Request

Property	Value
Endpoint	`POST /support`
Description	Send support email to development team
Authentication	Not required

Error Responses:

Code	Reason
422	Validation error in request body

1.11.11 Standard Error Response Format

All API endpoints follow a consistent error response structure:

HTTP Code	Description
200	Request succeeded
400	Bad request (invalid parameters or data)
401	Unauthorized (missing or invalid authentication token)
404	Resource not found
422	Unprocessable entity (validation error)
500	Internal server error

Generic Error Response Structure:

```
{
  "detail": "Human-readable error message"
}
```

1.12 Deployment

1.12.1 Deployment Architecture

The Full SMS platform deploys the frontend, backend API, and plugin execution service on a single AWS Lightsail instance, alongside managed Supabase services for database, auth, and storage.

1.12.2 Infrastructure Components

Frontend, Backend & Plugin Execution - AWS Lightsail

- **Platform:** Amazon Web Services (AWS Lightsail)
- **Instance:** Ubuntu, eu-west-3 (London), 4GB RAM, 2 vCPUs, 80GB SSD
- **Frontend:** Next.js, managed by PM2, port 3000
- **Backend API:** FastAPI (Python), Dockerized, port 8000
- **Plugin Execution:** FastAPI (Python), managed via `nohup`, internal port 8001
- **Task Queue:** Celery worker (Docker)
- **Message Broker/Cache:** Redis (Docker), port 6379
- **Reverse Proxy:** Nginx handles SSL termination (Let's Encrypt) and routes `/` to the frontend, `/api/py/*` to the backend
- Automatic CI/CD from GitHub Actions

Database & Storage - Supabase

- **Platform:** Supabase (managed PostgreSQL)
- PostgreSQL database with Row-Level Security (RLS)
- Object storage for HDF5 file uploads
- Real-time subscriptions for live updates

1.12.3 Deployment Diagram

The following diagram represents the **Production** environment deployment topology.

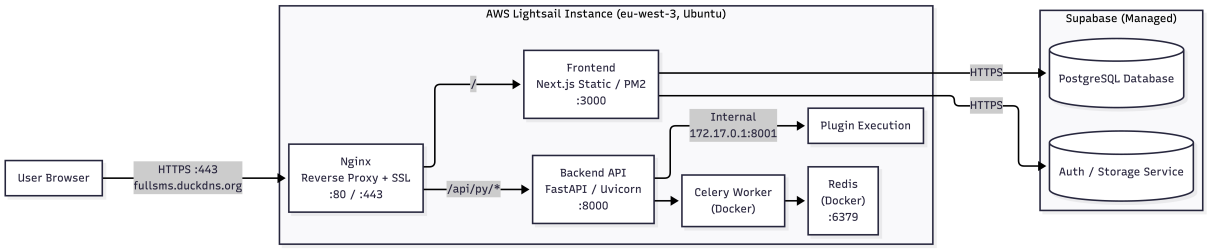

Figure 2: Full SMS Production Deployment Topology (AWS Lightsail + Supabase)

Nginx on the Lightsail instance terminates SSL and routes incoming HTTPS traffic to either the frontend (`/`) or the backend API (`/api/py/*`). The backend communicates with the internal plugin execution process and with Supabase over encrypted connections.

1.12.4 Node and Artifact Summary

Provider	Node	Artifacts	Access
AWS Lightsail	Ubuntu Instance	Nginx, Next.js (PM2), FastAPI (Docker), Plugin service (nohup), Celery, Redis	HTTPS :443
Supabase	Managed PostgreSQL	Database schema, Object storage	TCP :5432

1.12.5 Communication Paths

Source	Destination	Protocol	Purpose
User Browser	Nginx (Lightsail)	HTTPS :443	Serve frontend & route API requests
Nginx	Frontend (localhost:3000)	HTTP	Internal routing
Nginx	Backend API (localhost:8000)	HTTP	Internal routing
Backend API	Plugin Execution (172.17.0.1:8001)	HTTP	Internal plugin execution
FastAPI	Supabase PostgreSQL	TCP :5432	Database queries
FastAPI	Supabase Storage	HTTPS	HDF5 file storage

1.12.6 Environment Scope

This diagram represents the **Production** environment.

Environment	Configuration
Development	All services via Docker Compose; localhost URLs; local PostgreSQL
Staging	No separate preview environment currently; changes are tested locally before merging to main
Production	Full topology as shown; Lightsail public IP; instance always running

1.12.7 Deployment Flow

1. Developer pushes code to GitHub repository
2. CI pipeline runs tests and linting
3. On merge to main branch:
   - GitHub Actions SSHes into the Lightsail instance and pulls the latest code
   - Backend is rebuilt via Docker Compose
   - Frontend is rebuilt (`npm build`) and restarted via PM2
   - Plugin execution service is restarted as part of the backend rebuild
4. Health checks verify deployment success

1.12.8 Environment Configuration

Environment	Purpose	URL Pattern
Development	Local development	localhost
Staging	Pre-production testing	N/A (tested locally)
Production	Live application	fullsms.duckdns.org

1.12.9 Security Considerations

- All traffic encrypted via HTTPS/TLS
- Environment secrets managed via platform-specific secret managers
- Plugin execution containers run with minimal privileges
- Network isolation between execution containers and main infrastructure
- API authentication via JWT tokens

1.13 CI/CD Pipeline

The Full SMS platform employs a comprehensive continuous integration and continuous deployment (CI/CD) architecture to automate code quality validation, testing, and deployment across both frontend and backend systems.

1.13.1 Pipeline Overview

The CI/CD infrastructure consists of three distinct GitHub Actions workflows that handle code quality, testing, and deployment for both frontend and backend components. All pipelines are triggered automatically on code changes to ensure consistent quality and rapid iteration.

Workflow Summary:

Workflow	Trigger	Purpose
backend-ci.yml	Push/PR	Backend linting, testing, coverage
frontend-ci.yml	Push/PR	Frontend linting, testing, build verification
lightsail-deploy.yml	Push to main	Backend **and** frontend deployment to AWS Lightsail (Docker rebuild + PM2 restart)

1.13.2 Complete CI/CD Architecture Diagram

The following diagram illustrates the end-to-end continuous integration and deployment flow. Backend and frontend CI still run as parallel lint/test jobs, but both now converge on a single Lightsail deployment workflow rather than separate Vercel and Lightsail deployments.

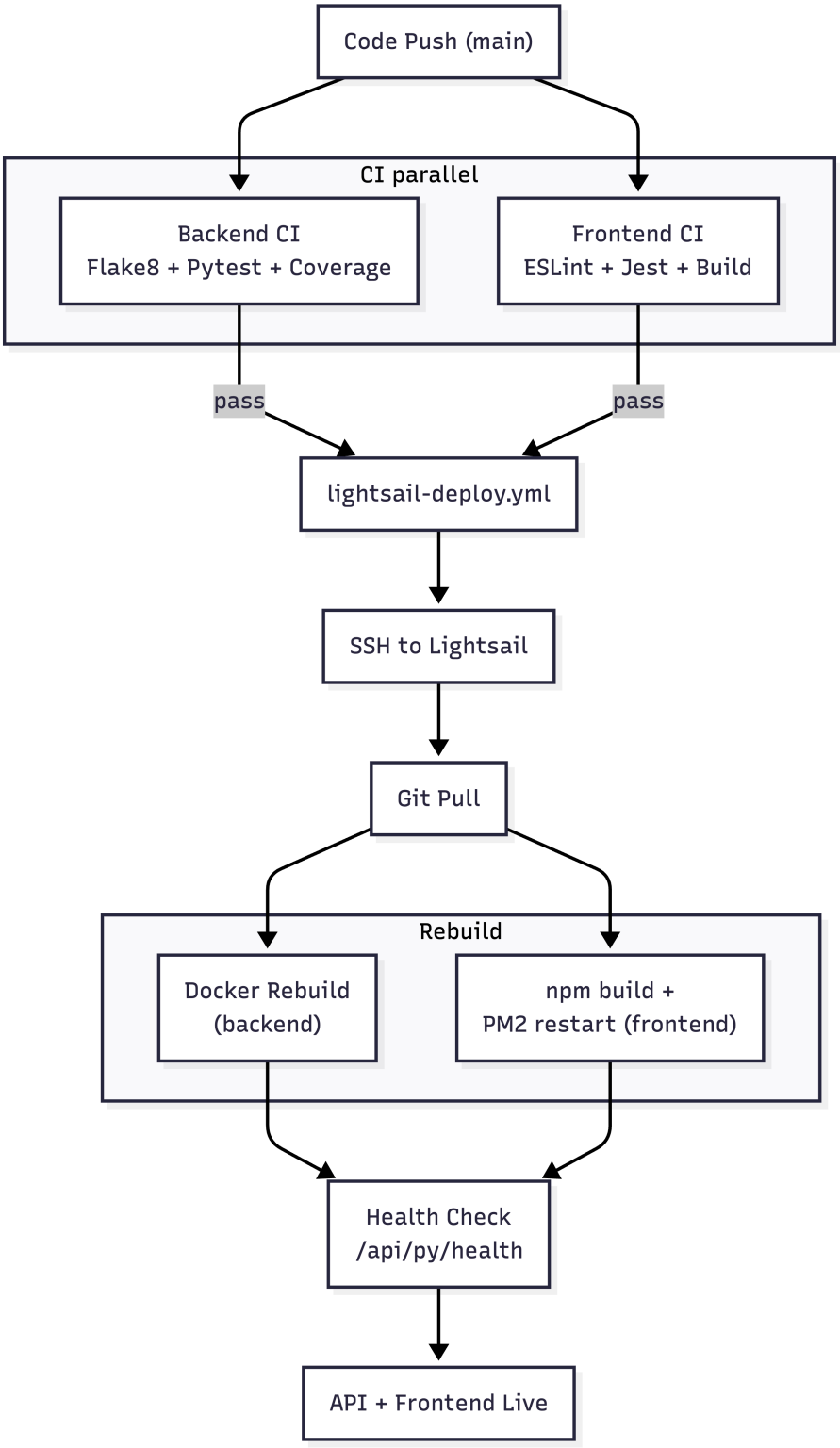

Figure 3: Consolidated CI/CD Pipeline: Code Push → CI (lint/test, backend and frontend in parallel) → SSH to Lightsail → Git Pull → Docker Rebuild (backend) + npm build & PM2 restart (frontend) → Health Check → API/Frontend Live

1.13.3 Continuous Integration Workflows

Backend CI (backend-ci.yml)

This workflow validates backend code quality and functionality on every push or pull request to the develop or main branches.

Stage	Actions
Environment Setup	Install Python 3.11, pip dependencies, flake8, pytest, pytest-cov
Code Linting	Run flake8 to detect syntax errors, undefined names, and code quality issues (max complexity: 10, max line length: 127)
Unit Testing	Execute pytest test suite with coverage tracking for all modules in api/tests/
Coverage Reporting	Upload coverage report to Codecov with backend flag for visibility

Test Environment Variables:

- Placeholder Supabase credentials for isolated testing
- Local Redis configuration (localhost:6379)
- Mock execution service endpoint
- CORS origins limited to localhost:3000

Frontend CI (frontend-ci.yml)

This workflow ensures frontend code quality, functionality, and build success on every push or pull request.

Stage	Actions
Environment Setup	Install Node.js 20, cache npm dependencies, run npm ci for clean install
Code Linting	Execute ESLint to enforce code style and detect potential errors
Unit Testing	Run Jest unit tests with coverage, excluding integration tests
Coverage Reporting	Upload test coverage to Codecov with frontend flag
Integration Testing	Execute integration test suite (continue on error for non-blocking validation)
Build Verification	Run production build to ensure Next.js compilation succeeds

Build Environment:

- Next.js production build targeting static and server-side rendering
- API URL configured for localhost:8000

- Supabase credentials from GitHub Secrets

1.13.4 Continuous Deployment Workflows

Backend Deployment (lightsail-deploy.yml)

Automated deployment workflow for backend API to AWS Lightsail infrastructure.

Stage	Actions
Trigger	Push to develop or main branch activates deployment
SSH Connection	GitHub Actions establishes SSH connection to Lightsail instance using ED25519 private key from secrets
Code Sync	Navigate to /Full-SMS directory, fetch origin, checkout target branch, pull latest changes
Container & Process Rebuild	Stop running backend containers (docker-compose down), rebuild images with latest code, restart in detached mode (docker-compose up -d –build); rebuild frontend (npm run build) and restart via pm2 restart
Health Verification	Wait 15 seconds for startup, perform HTTP health check at /api/py/health with 6 retry attempts (5-second intervals); verify frontend responds on port 3000
Notification	Log deployment success or failure with commit details and author information

Required GitHub Secrets:

- LIGHTSAIL_HOST: Public IP address of AWS instance
- LIGHTSAIL_USER: SSH username (ubuntu)
- LIGHTSAIL_SSH_KEY: ED25519 private key for passwordless authentication

Note: VERCEL_ORG_ID, VERCEL_PROJECT_ID, and VERCEL_TOKEN are no longer needed and can be removed from repository secrets.

1.13.5 Pipeline Execution Matrix

Workflow	Trigger Event	Branches	Path Filter	Runner
Backend CI	Push, PR	develop, main	api/**	ubuntu-latest
Frontend CI	Push, PR	develop, main	frontend/**	ubuntu-latest
Lightsail Deploy	Push	develop, main	api/**, frontend/**	ubuntu-latest

1.13.6 Environment Strategy

The CI/CD pipeline implements a multi-environment deployment strategy to support development, staging, and production workflows.

Environment	Branch	Frontend URL	Backend
Development	Local branches	localhost:3000	localhost:8000
Staging	develop	N/A (tested locally)	Lightsail instance
Production	main	https://fullsms.duckc	Lightsail instance

1.13.7 Quality Gates

All code changes must pass the following quality gates before deployment:

1. **Linting:** Code must pass static analysis (flake8 for Python, ESLint for TypeScript)
2. **Unit Tests:** All unit tests must execute successfully with no failures
3. **Coverage:** Test coverage reports uploaded to Codecov for tracking (no minimum threshold enforced)
4. **Build:** Frontend build must complete without errors
5. **Health Check:** Deployed backend must respond to health endpoint within timeout period

1.13.8 Rollback Strategy

Backend Rollback:

- Health check failures prevent deployment completion, leaving previous containers running
- Manual rollback via git revert and re-trigger deployment workflow
- Docker image caching enables rapid redeployment of previous versions
- SSH access allows manual intervention for emergency fixes

Frontend Rollback:

- Manual rollback via git revert and re-trigger of the deployment workflow
- PM2 restart applies the reverted build; brief downtime may occur during the restart
- Previous build artifacts can be restored from GitHub commit history if needed

1.13.9 Monitoring and Observability

- **GitHub Actions Logs:** Complete execution logs for all workflow runs with step-by-step output
- **Codecov Dashboard:** Historical coverage trends, file-level coverage reports, and pull request coverage diffs
- **PM2 Logs:** Frontend process logs, restart history, and resource usage (pm2 logs, pm2 monit)
- **Lightsail Metrics:** CPU utilization, network traffic, and disk usage via AWS CloudWatch
- **Docker Logs:** Container-level logs accessible via docker-compose logs command

1.13.10 Security Considerations

- All sensitive credentials stored in GitHub encrypted secrets vault
- SSH authentication using ED25519 key pairs instead of passwords
- Secrets never exposed in workflow logs or build outputs
- Environment variables managed via .env files on the Lightsail instance, not committed to version control
- Minimal privilege principle: deployment keys restricted to specific operations
- HTTPS enforcement for all external communication
- Regular secret rotation recommended every 90 days